

君正®

IMPP 说明文档

Date: Dec. 2022



北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®

IMPP说明文档

Copyright © Ingenic Semiconductor Co. Ltd 2022. All rights reserved.

Release history

Date	Revision	Change
Jan.2022	1.00	First release
Dec.2022	2.00	Second release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

Ingenic Semiconductor Co., Ltd.

Ingenic Headquarters, East Bldg. 14, Courtyard #10
Xibeiwang East Road, Haidian District, Beijing, China,
Tel: 86-10-56345000
Fax:86-10-56345001
Http: //www.ingenic.com

北京君正集成电路股份有限公司

地址:北京市海淀区东北西路中关村软件园二期君正总部大楼
电话: 86-10-56345000
传真: 86-10-56345001
http: //www.ingenic.com

目 录

1 IMPP 简介	1
2 Camera 模块	2
2.1 Camera 模块相关 API 介绍	2
2.2 ISP 缩放裁剪属性说明	7
2.3 使用 DMA-BUF 的内存共享	8
2.4 使用示例	9
3 Display 模块	10
3.1 Display 模块 API 介绍	10
3.2 Display 模块使用流程	14
3.3 使用示例	15
4 DPU 模块	16
4.1 DPU 模块 API 介绍	17
4.2 使用示例	20
5 Rotater 模块	21
5.1 Rotater 模块 API 介绍	21
5.2 Rotater 的 DMA-BUF 内存共享	25
5.3 使用示例	26
6 Codec 模块	27
6.1 Codec 模块 API 介绍	27
6.2 使用示例	35
7 Audio 模块	37
7.1 Audio 模块 API 介绍	37
7.2 使用示例	52
8 IPU 模块	53
8.1 IPU 模块 API 介绍	53
8.2 使用示例	60
9 DrawBox 模块	61
9.1 DrawBox 模块 API 介绍	61
9.2 使用示例	62

关 键 词	
摘 要	
缩 略 语	
参考资料	

1 IMPP 简介

IMPP(Ingenic-Media-Process-Platform)，可进行音视频采集播放、音视频编解码以及图像的 2D 处理(旋转/透明叠加/格式转换)。

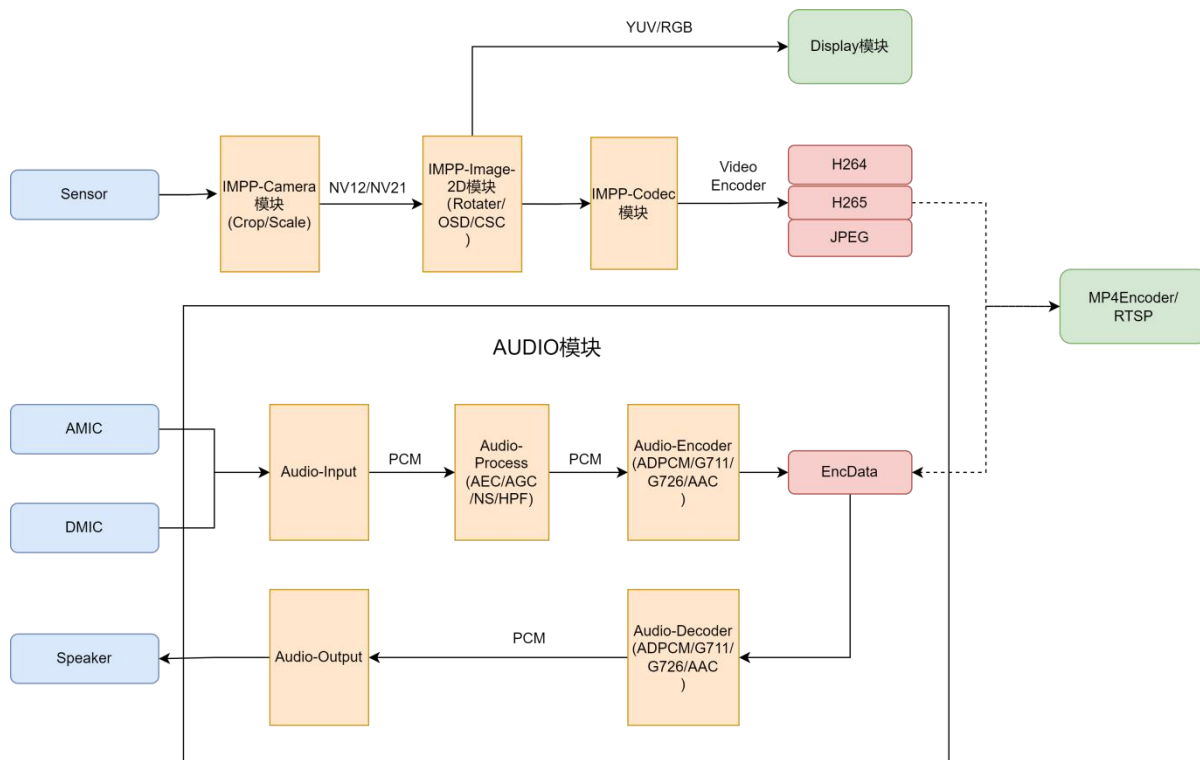


图 1-1 IMPP 结构

IMPP 主要包含：Camera 模块、IPU 模块 (OSD/CSC)、Rotater 模块 (图像旋转)、Codec 模块 (视频编解码)、Display 模块、DPU 模块、Audio 模块；各模块间可通过 Dma-buf 进行内存共享，实现零拷贝，从而提升效率。

X2000 和 X2500 目前支持功能列表：

功能模块	X2000/M300/X2100	X2500
Audio 模块 (采集、播放、编码、解码、音频处理)	除 AEC-API 不支持，其它均支持	所有 API 均支持
Camera 模块	所有 API 均支持	所有 API 均支持
Video-ENC 模块 (Codec)	所有 API 均支持	所有 API 均支持
Display 模块	所有 API 均支持	所有 API 均支持
DPU	所有 API 均支持	所有 API 均支持
Rotater 模块	所有 API 均支持	所有 API 均支持
IPU (CSC/OSD) 模块	不支持	所有 API 均支持
Drawbox 模块	不支持	所有 API 均支持

2 Camera 模块

Camera 模块实现摄像头数据采集，X2000/X2500 芯片平台可支持输出 NV12/NV21 格式。X2000/X2500 芯片平台的视频采集模块，在驱动实现方式上均是基于 V4L2 框架，下表列出各视频节点和 Sensor 的对应关系。

Sensor N	设备节点	输出格式
Sensor0	/dev/video3	RAW/YUV (未经 ISP 处理)
	/dev/video4	NV12/NV21
	/dev/video5	NV12/NV21
	/dev/video6	NV12/NV21
Sensor1	/dev/video7	RAW/YUV (未经 ISP 处理)
	/dev/video8	NV12/NV21
	/dev/video9	NV12/NV21
	/dev/video10	NV12/NV21
Sensor2 (DVP)	/dev/video13	

需要注意的是，X2500 ISP 最大输出分辨率为 3840*2160，X2000 ISP 最大输出分辨率为 2048*2048。

2.1 Camera 模块相关 API 介绍

2.1.1 打开视频输入节点

【函数原型】

```
IHAL_CameraHandle_t* IHal_CameraOpen(IHAL_INT8* videoname);
```

【功能描述】

打开一个视频输入节点

【API 参数说明】

参数	描述	输入/输出
videoname	视频节点路径	输入

【返回值】

成功: Camera Handle

失败: NULL

【注意事项】

无。

2.1.2 关闭视频输入节点

【函数原型】

```
IHAL_INT32 IHal_CameraClose(IHAL_CameraHandle_t* handle);
```

【功能描述】

关闭一个已打开的视频输入节点。

【API 参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

2.1.3 设置节点参数

【函数原型】

```

IHAL_INT32 IHal_CameraSetParams(IHAL_CameraHandle_t* handle,
                                IHAL_CAMERA_PARAMS* params);

```

【功能描述】

设置 handle 对应节点的视频参数，主要是图像宽高和像素格式等。

【API 参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
params	图像参数结构	输入

【ISP 输出参数说明】

参数	描述
imageWidth	ISP 输出图像宽度
imageHeight	ISP 输出图像高度
imageFmt	ISP 输出像素格式
imageFmtStr	图像像素格式字符串，当 imageFmtStr 不为 NULL 时，以 imageFmtStr 为主
wdr_enable	是否开启 WDR
bin_select	是否指定 BIN 文件，不指定则使用默认文件路径
crop_posx	ISP-CORE 输出裁剪起始 X 坐标
crop_posy	ISP-CORE 输出裁剪起始 Y 坐标
crop_width	ISP-CORE 输出裁剪宽度
crop_height	ISP-CORE 输出裁剪高度
scale_out_crop_posx	缩放输出裁剪起始坐标 X
scale_out_crop_posy	缩放输出裁剪起始坐标 Y
scale_out_crop_width	缩放输出裁剪宽度
scale_out_crop_height	缩放输出裁剪高度
binpath	指定 BIN 文件的路径(64Byte)

【返回值】

成功：0

失败：非 0

【注意事项】

无。

2.1.4 设置 ISP 控制参数

【函数原型】

```
IHAL_INT32 IHal_CameraSetControlParams(IHAL_CameraHandle_t* handle,  
                                         IHAL_CAMERA_CONTROL_CMD cmd, IHAL_UINT32 value);
```

【功能描述】

设置 handle 对应节点的视频控制参数，如白平衡、自动曝光等参数。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
cmd	要设置的参数对应的 CMD	输入
value	参数值	输入

【返回值】

成功：0

失败：非 0

【注意事项】

设置 ISP 控制参数必须在 Camera 开流之后进行设置，即在 IHal_CameraStart()之后进行调用。

2.1.5 获取 ISP 控制参数

【函数原型】

```
IHAL_INT32 IHal_CameraGetControlParams(IHAL_CameraHandle_t* handle,  
                                         IHAL_CAMERA_CONTROL_CMD cmd, IHAL_UINT32 *value);
```

【功能描述】

获取 handle 对应节点的视频控制参数，如白平衡、自动曝光等参数。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
cmd	要获取的参数对应的 CMD	输入
value	参数值	输出

【返回值】

成功：0

失败：非 0

2.1.6 创建 Buffer

【函数原型】

```
IHAL_INT32 IHal_CameraCreateBuffers(IHAL_CameraHandle_t* handle,  
                                     IMPP_BUFFER_TYPE type, IHAL_INT32 num_buffers);
```

【功能描述】

创建 camera 的数据 buffer。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
type	需要创建 buffer 的类型，通常有三种类型： IMPP_INTERNAL_BUFFER: 内部 Buffer; IMPP_EXT_DMABUFFER: 外部的 dma-buf。 IMPP_EXT_USERBUFFER: 外部的用户空间 buffer, 此用户空间 Buffer。	输入
num_buffers	需要创建 buffer 的个数	输入

【返回值】

成功: 创建 buffer 的个数

失败: -1

2.1.7 设置共享 Buffer

【函数原型】

```

IHAL_INT32 IHal_CameraSetBuffers(IHAL_CameraHandle_t* handle,
                                   IHAL_INT32 index, IMPP_BufferInfo_t *sharebuf);

```

【功能描述】

设置 camera 的数据缓冲 buffer，该 API 用于 camera 使用外部数据 Buffer 时调用，需要在 camera 开始前调用，且只需要调用一次。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
index	buffer 的序号	输入
sharebuf	内存共享描述结构体	输入

【返回值】

成功: 0

失败: 非 0

2.1.8 获取共享 Buffer

【函数原型】

```

IHAL_INT32 IHal_GetCameraBuffers(IHAL_CameraHandle_t* handle,
                                   IHAL_INT32 index, IMPP_BufferInfo_t *sharebuf);

```

【功能描述】

获取 camera 的数据 buffer，该 API 主要用于将 camera 的内部申请的 buffer 和其他模块的共享(如编码器)，从而实现数据的零拷贝。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
index	buffer 的序号	输入
sharebuf	内存共享描述结构体	输出

【返回值】

成功: 0

失败: 非 0

2.1.9 开启视频流输出

【函数原型】

```
IHAL_INT32 IHal_CameraStart(IHAL_CameraHandle_t* handle);
```

【功能描述】

开启 Camera 节点视频流输出。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入

【返回值】

成功: 0

失败: 非 0

2.1.10 停止视频流输出

【函数原型】

```
IHAL_INT32 IHal_CameraStop(IHAL_CameraHandle_t* handle);
```

【功能描述】

Camera 节点停止流输出。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入

【返回值】

成功: 0

失败: 非 0

2.1.11 等待 Camera 数据

【函数原型】

```
IHAL_INT32 IHal_Camera_WaitBufferAvailable(IHAL_CameraHandle_t* handle,  
                                             IHAL_INT32 impp_wait);
```

【功能描述】

等待 Camera-buffer 为可用状态, 即图像数据已填充完成。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
impp_wait	等待类型, 阻塞等待 (IMPP_WAIT_FOREVER) 和非阻塞等待 (IMPP_NO_WAIT)	输入

【返回值】

成功: 0

失败: 非 0

2.1.12 获取 Camera 数据 Buffer

【函数原型】

```
IHAL_INT32 IHal_CameraDeQueueBuffer(IHAL_CameraHandle_t* handle,  
                                      IMPP_BufferInfo_t *buf);
```

【功能描述】

Camera-Buffer 出队列，从 camera 驱动获取一个已填充的数据 buffer 到应用层。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
bufinfo	Camera 数据 buffer 描述	输出

【返回值】

成功：0

失败：非 0

2.1.13 释放 Camera 数据 Buffer

【函数原型】

```
IHAL_INT32 IHal_CameraQueuebuffer(IHAL_CameraHandle_t* handle,  
                                    IMPP_BufferInfo_t *buf);
```

【功能描述】

Camera-Buffer 入队列，应用层使用完 buffer 后，将 Buffer 放回 camera 驱动。

【参数说明】

参数	描述	输入/输出
handle	Camera 句柄	输入
buf	Camera 数据 buffer 信息	输入

【返回值】

成功：0

失败：非 0

2.2 ISP 缩放裁剪属性说明

Camera 模块可支持摄像头图像裁剪和缩放，X2000 仅支持缩放后的裁剪（Scale-Out-Crop）；X2500 支持 ISP 输出的裁剪（crop）和缩放输出的裁剪（Scale-Out-Crop）。裁剪属性可通过在设置参数时进行相应的配置。缩放裁剪的底层硬件工作流程如图所示：

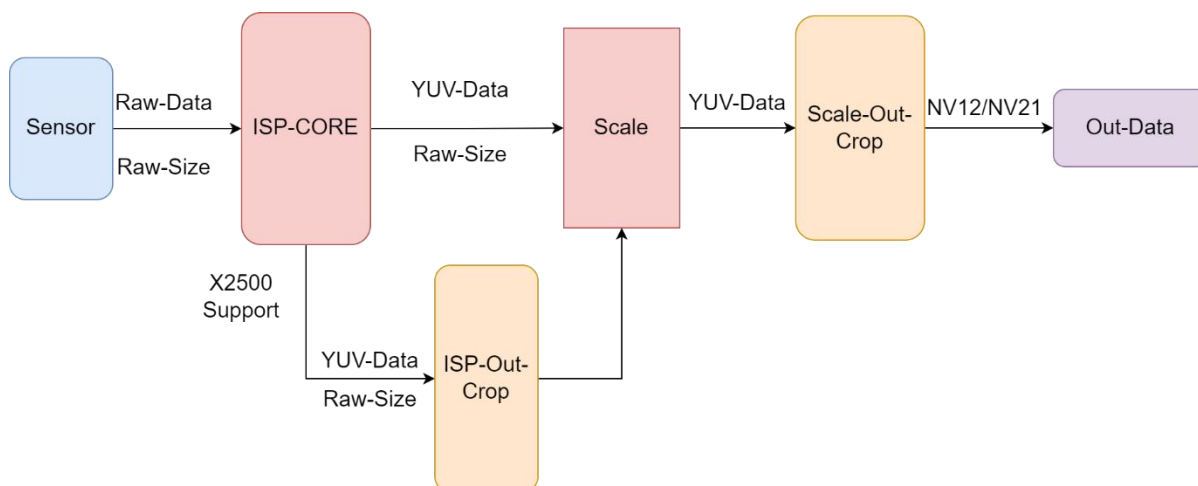


图 2-1 ISP 裁剪缩放

2.3 使用 DMA-BUF 的内存共享

Camera 模块可支持 DMA-BUF 的导入和导出。导出，将 Camera 申请的 Buffer 导出为 fd, 用于和其它模块的内存共享，从而达到零拷贝的目的。导入，将外部的 dma-buf 的 fd 作为 Camera 模块的数据 buffer，也能达到内存共享的目的。

举例一，将 Camera 模块 buffer 导出给其它模块使用：

```

IHAL_CameraHandle_t *handle = IHal_CameraOpen("/dev/video4");

int ret = IHal_CameraSetParams(handle,&params);

/* 这里 Camera 申请内部 buffer, 用于后续的导出共享 */
ret = IHal_CameraCreateBuffers(handle,IMPP_INTERNAL_BUFFER,2);
/* 导出 dma-buffer */
for(int i = 0; i < 2; i++){
    ret = IHal_GetCameraBuffers(camera_handle,i,&share);
    printf("fd = %d size = %d \r\n",share.fd,share.size);
    ret = IHal_Codec_SetSrcBuffer(handle,i,&share);
}
  
```

举例二，其它模块将 dma-buf 导入 Camera 模块

```

IHAL_CameraHandle_t *handle = IHal_CameraOpen("/dev/video4");

int ret = IHal_CameraSetParams(handle,&params);

/* 这里 Camera 申请外部 DMA-buffer, 后续的导出共享 */
ret = IHal_CameraCreateBuffers(handle,IMPP_EXT_DMABUFFER,2);
/* 导入 dma-buffer */
for(int i = 0; i < 2; i++){
    IHal_Rot_SrcDmaBufGet(rot_handle,&share,i);
}
  
```

```
ret = IHal_CameraSetBuffers(camera_handle,i,&share);  
printf("fd = %d size = %d \r\n",share.fd,share.size);  
}
```

需要注意的是，IHal_CameraDeQueueBuffer，从 Camera 获取数据 buffer 并占据使用权，在未使用完之前不能调用 IHal_CameraQueuebuffer，在使用完之后，必须调用 IHal_CameraQueuebuffer，将 buffer 使用权给到 Camera。

2.4 使用示例

Camera 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/camera-example

代码功能说明：

代码文件	功能
camera_example.c	拍摄一张图片并保存，默认图像像素格式是 NV12

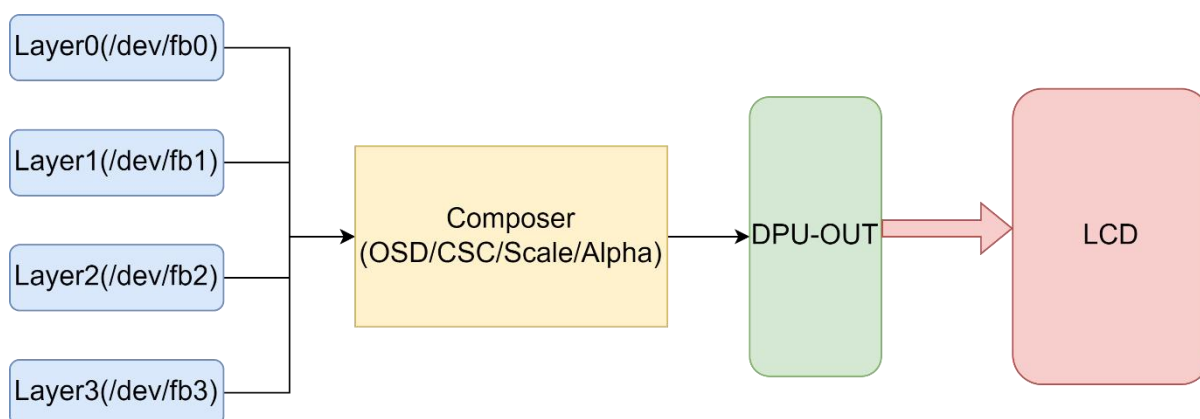
3 Display 模块（DPU Direct-out）

Display 模块，结合驱动的设计实现了对 framebuffer 操作的封装，便于开发者使用。Display 模块分为 RDMA 模式和 Composer 模式，RDMA 模式只支持单层显示，且只能显示 RGB 格式，适用于较为简单的应用场景；Composer 模式，最大可支持四层图像的叠加、两层图像的缩放(任意两层)，其中 0 层和 1 层可支持 YUV 数据和 RGB 数据，第 2 层和 3 层只支持 RGB 数据。Composer 模式下可支持的格式具体为：

YUV	NV12 / NV21
RGB	RGB888 / ARGB888 / RGB555 / ARGB1555 / RGB565

内核默认配置为 Composer 模式，如需要使用 RDMA 模式，需要对设备树进行相应的修改。具体修改方式可参考内核开发相关文档。

Composer 模式下，硬件 DPU 的工作示意：



3.1 Display 模块 API 介绍

3.1.1 Framebuffer 初始化

【函数原型】

```
IHal_SampleFB_Handle_t* IHal_SampleFB_Init(IHal_SampleFB_Attr *attr);
```

【功能描述】

初始化一个设备节点，对应 DPU 的一个 Layer。

【参数说明】

参数	描述	输入/输出
attr	属性参数	输入

【返回值】

成功：handle

失败：NULL

【注意事项】

无。

3.1.2 Framebuffer 反初始化

【函数原型】

```
IHAL_INT32 IHal_SampleFB_DeInit(IHal_SampleFB_Handle_t *handle);
```

【功能描述】

反初始化一个设备节点，对应 DPU 的一个 Layer。

【参数说明】

参数	描述	输入/输出
handle	需要关闭的 handle	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.3 获取 Framebuffer-MEM 信息

【函数原型】

```
IHAL_INT32 IHal_SampleFB_GetMem(IHal_SampleFB_Handle_t *handle,  
                                  IMPP_BufferInfo_t *buf);
```

【功能描述】

获取 FB 的 MEM 信息，可获取到 Buffer 的物理地址、虚拟地址以及大小。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
buf	Buffer 信息	输出

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.4 更新数据显示

【函数原型】

```
IHAL_INT32 IHal_SampleFB_Update(IHal_SampleFB_Handle_t *handle,  
                                  IMPP_BufferInfo_t *buffer);
```

【功能描述】

更新数据显示，通常在将数据写到 Buffer 后，需要调用。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
buf	Buffer 信息	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.5 设置显示源数据大小

【函数原型】

```
IHAL_INT32 IHal_SampleFB_SetSrcFrameSize(IHal_SampleFB_Handle_t *handle,  
                                           IHAL_INT32 width, IHAL_INT32 height);
```

【功能描述】

设置源数据大小，若源数据大小和目标大小不一致，则会进行缩放。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
width	源数据宽度	输入
height	源数据高度	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.6 设置源 FrameBuffer 裁剪信息

【函数原型】

```
IHAL_INT32 IHal_SampleFB_SetSrcCrop(IHal_SampleFB_Handle_t *handle,  
                                       IHAL_INT32 crop_x, IHAL_INT32 crop_y,  
                                       IHAL_INT32 crop_w, IHAL_INT32 crop_h);
```

【功能描述】

设置源 FrameBuffer 裁剪信息。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
crop_x	起始坐标 X	输入
crop_y	起始坐标 Y	输入
crop_w	裁剪的宽度	输入
crop_h	裁剪的高度	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.7 设置显示的目标大小

【函数原型】

```
IHAL_INT32 IHal_SampleFB_SetTargetFrameSize(IHal_SampleFB_Handle_t *handle,  
                                              IHAL_INT32 width, IHAL_INT32 height);
```

【功能描述】

设置显示目标大小，若目标大小和源数据大小不一致，则会进行缩放。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
width	目标宽度	输入
height	目标高度	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.8 设置显示目标位置

【函数原型】

```
IHAL_INT32 IHal_SampleFB_SetTargetPos(IHal_SampleFB_Handle_t *handle,  
                                        IHAL_INT32 posx, IHAL_INT32 posy);
```

【功能描述】

设置显示目标位置。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
posx	目标位置横坐标	输入
posy	目标位置纵坐标	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.9 设置图层叠加顺序

【函数原型】

```
IHAL_INT32 IHal_SampleFB_SetZorder(IHal_SampleFB_Handle_t *handle,  
                                     FB_Composer_Zorder_t order);
```

【功能描述】

设置图层叠加顺序。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
order	叠加顺序	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.10 更新参数设置

【函数原型】

```
IHAL_INT32 IHal_SampleFB_CompRestart(IHal_SampleFB_Handle_t *handle);
```

【功能描述】

重启 Composer，更新参数设置。

【参数说明】

参数	描述	输入/输出
handle	handle	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.1.11 使能或禁用对应的 Layer

【函数原型】

```
IHAL_INT32 IHal_SampleFB_Layer_Enable(IHal_SampleFB_Handle_t *handle, int enable);
```

【功能描述】

使能或禁用对应的 Layer。

【参数说明】

参数	描述	输入/输出
handle	handle	输入
enable	使能位 (0 或 1)	输入

【返回值】

成功:0

失败: 非 0

【注意事项】

无。

3.2 Display 模块使用流程

```
IHal_SampleFB_Attr fb_attr;
memset(&fb_attr, 0, sizeof(IHal_SampleFB_Attr));
```

```

sprintf(&fb_attr.node[0], "%s", "/dev/fb0");
fb_attr.mode = Composer_Mode;
fb_attr.frame_width = 720;
fb_attr.frame_height = 1280;
fb_attr.alpha = 255;
fb_attr.frame_fmt = IMPP_PIX_FMT_NV12; /* 指定 Layer 格式 */
IHal_SampleFB_Handle_t *fb0_handle = IHal_SampleFB_Init(&fb_attr);
if(!fb0_handle){
    printf("init error");
    return -1;
}
/* 设置源图像大小 */
IHal_SampleFB_SetSrcFrameSize(fb0_handle, 720, 1280);
/* 设置目标图像大小, 如果和源图像大小不等, 则会进行缩放 */
IHal_SampleFB_SetTargetFrameSize(fb0_handle, 720, 1280);
/* 指定显示起始位置 */
IHal_SampleFB_SetTargetPos(fb0_handle, 0, 0);
/* 指定 Layer 的叠加顺序 */
IHal_SampleFB_SetZorder(fb0_handle, Order_0);
/* 动态调整的参数需要配合 IHal_SampleFB_CompRestart() 更新配置. */
IHal_SampleFB_CompRestart(fb0_handle);

IMPP_BufferInfo_t fb0_buf;
/* 获取 Framebuffer Mem 信息, 主要为物理地址, 虚拟地址, Buffer 大小 */
IHal_SampleFB_GetMem(fb0_handle, &fb0_buf);
/* 填充 framebuffer 后, 更新显示 */
/* 去初始化 */
IHal_SampleFB_Update(fb0_handle, &fb0_buf);

/* 去初始化 */
IHal_SampleFB_DeInit(fb0_handle);

```

3.3 使用示例

Display 测试程序位于 SDK 根目录下:

development/Ingenic-Mpp/test/display-example

代码功能说明:

代码文件	功能
sample_lcd_comp_test.c	通过 LCD 两层缩放并显示 Camera 的图像
sample_lcd_comp_usrptr_test.c	通过 LCD 两层缩放并显示双路 Camera 图像数据

4 DPU 模块（DPU RDMA+Composer）

DPU 模块用于 OSD 图像叠加功能，OSD 图像叠加功能是通过 Composer 模式实现的，因此还需将 Layer 使能并导出至少两个，两层以上的叠加需要更多的导出。Composer 模式最大可支持四层图像的叠加、两层图像的缩放（任意两层），其中 0 层和 1 层可支持 YUV 数据和 RGB 数据，第 2 层和第 3 层只支持 RGB 数据。Composer 模式下可支持的格式具体为：

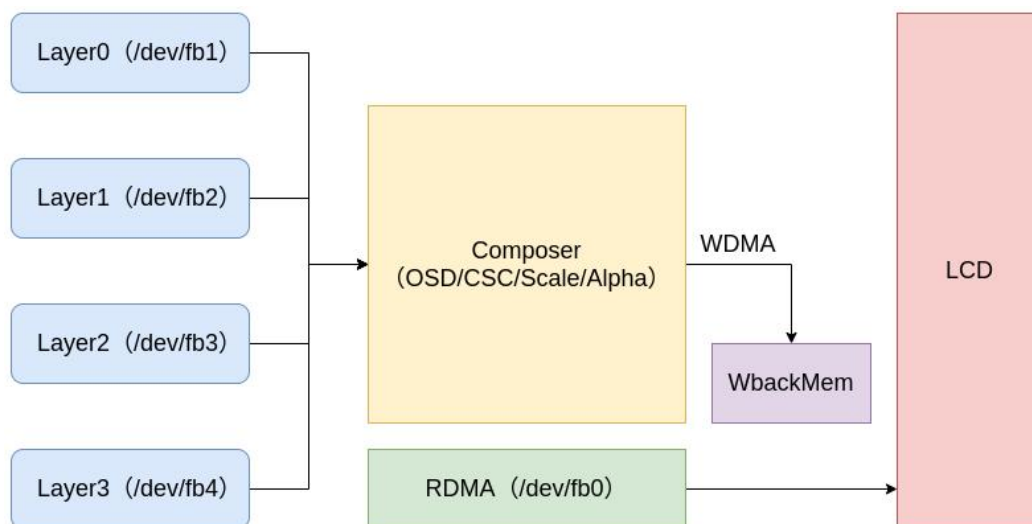
YUV	NV12 / NV21
RGB	RGB888 / ARGB8888 / RGB555 / ARGB1555 / RGB565

内核默认配置为 Composer 模式，使用 DPU 模块时需要将其配置成 RDMA 模式，具体的修改方式可参考内核开发相关文档。

DPU 模块支持将 Composer 输出的图像数据通过 WDMA 写回至 memory，写回格式具体支持为：

YUV	不支持
RGB	ARGB8888 / RGB555 / RGB565 / RGB888

DPU 模块 OSD 图像叠加功能的硬件工作示意图：



[注]：

1. WDMA : Write back DMA。
2. RDMA : Read DMA

4.1 DPU 模块 API 介绍

4.1.1 初始化 DPU 模块

【函数原型】

```
IHal_Dpu_Handle_t* IHal_Dpu_Init(IHal_Dpu_InitStruct_t *init);
```

【功能描述】

DPU 功能初始化。

【参数说明】

参数	描述	输入/输出
init	初始化参数	输入

【返回值】

成功: DPU handle

失败: NULL

【注意事项】

无。

4.1.2 去初始化 DPU 模块

【函数原型】

```
IHAL_INT32 IHal_Dpu_Deinit(IHal_Dpu_Handle_t *handle);
```

【功能描述】

DPU 功能去初始化。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

4.1.3 获取 RDMA 的 Buffer 数量

【函数原型】

```
IHAL_INT32 IHal_Dpu_RDMA_Get_BufferNums(IHal_Dpu_Handle_t* handle);
```

【功能描述】

获取用于屏幕显示的 Buffer 数量 (RDMA 的 buffer 数量)。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入

【返回值】

成功: RDMA 的 Buffer 数量

失败: -1

【注意事项】

无。

4.1.4 获取 RDMA 的帧 Buffer

【函数原型】

```

IHAL_INT32 IHal_Dpu_RDMA_GetFrame(IHal_Dpu_Handle_t* handle,
                                     IMPP_BufferInfo_t *buf);

```

【功能描述】

从 RDMA 获取帧 Buffer，可获取到帧 Buffer 的物理地址、虚拟地址以及大小等。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入
buf	帧 Buffer 信息	输出

【返回值】

成功：0

失败：非 0

【注意事项】

无。

4.1.5 将帧 Buffer 归还给 RDMA 硬件，用于上屏显示

【函数原型】

```

IHAL_INT32 IHal_Dpu_RDMA_PutFrame(IHal_Dpu_Handle_t* handle, IMPP_BufferInfo_t
*buf);

```

【功能描述】

将帧 Buffer 归还给 RDMA 硬件，用于上屏显示。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入
buf	帧 Buffer 信息	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

4.1.6 设置输出数据 Buffer

【函数原型】

```

IHAL_INT32 IHal_Dpu_Composer_Set_WbackBuffers(IHal_Dpu_Handle_t* handle,
IMPP_BufferInfo_t *buf,IHAL_INT32 index);

```

【功能描述】

设置 DPU(OSD/CSC)输出数据 Buffer，支持 DMA-Buf。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入

buf	Buffer 信息	输入
index	Buffer 的编号	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

4.1.7 DPU 进行图像处理

【函数原型】

```
IHAL_INT32 IHal_Dpu_Composer_Process(IHal_Dpu_Handle_t* handle,  
                                       IHal_Dpu_FrameDesc_t *frame);
```

【功能描述】

DPU(OSD/CSC)进行一次图像处理，更新 Composer 配置。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入
frame	帧描述符	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

4.1.8 获取输出数据

【函数原型】

```
IHAL_INT32 IHal_Dpu_Composer_GetFrame(IHal_Dpu_Handle_t* handle,  
                                       IMPP_FrameInfo_t *frame);
```

【功能描述】

获取 DPU 处理输出数据，获取后模块则没有该 buffer 的使用权。Writeback 使能时，可以获取 composer 叠加后的数据。

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入
frame	输出数据描述	输出

【返回值】

成功：0

失败：非 0

【注意事项】

无。

4.1.9 释放输出数据

【函数原型】

```
IHAl_INT32 IHal_Dpu_Composer_ReleaseFrame(IHal_Dpu_Handle_t* handle,  
                                           IMPP_FrameInfo_t *frame);
```

【功能描述】

释放输出数据，writeback 使能时，将 frame 的使用权交还给硬件。（与 IHal_Dpu_Composer_GetFrame() 配套使用）

【参数说明】

参数	描述	输入/输出
handle	DPU handle	输入
frame	输出数据描述	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

4.2 使用示例

DPU 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/display-example

代码功能说明：

代码文件	功能
dpu_osd_test.c	DPU 用于 OSD 图像叠加功能并循环地改变 DPU 输入信息
dpu_camera_test.c	一路 Camera 数据经缩放到叠加到另一路 Camera 数据
camera_pic_dpu_osd_switch_order.c	每 2s 互换 Camera 与图片的 Order

5 Rotater 模块

Rotater 模块支持图像的 0、90、180、270 度的旋转，支持上下翻转和左右镜像，需要注意的是，旋转和翻转镜像不能同时操作，如需旋转+翻转或旋转+镜像，则需要分两步进行。

不同芯片平台支持的格式如下：

芯 片 平台	输入格式	输出格式
X2000	ARGB8888/RGB888/RGB565/RGB555/RGB1555/YUV422	ARGB8888/RGB565/RGB555/YUV422
X2500	NV12/RAW8/RGB565/ARGB8888	NV12/RAW8/RGB565/ARGB8888

需要注意的是 X2500 输入和输出格式必须保持一致。

5.1 Rotater 模块 API 介绍

5.1.1 创建图像旋转通道

【函数原型】

```
IHal_Rot_Handle_t* IHal_Rot_CreateChan(IHal_Rot_ChAttr_t *attr);
```

【功能描述】

创建图像旋转通道，最多可创建 4 个独立通道。

【参数说明】

参数	描述	输入/输出
attr	属性参数	输入

【返回值】

成功：handle

失败：NULL

【注意事项】

图像旋转模块，可支持共享 buffer，从而做到数据的零拷贝，在通道创建时需要指定 SRC 和 DST Buffer 的类型。

5.1.2 销毁图像旋转通道

【函数原型】

```
IHAL_INT32 IHal_Rot_DestroyChan(IHal_Rot_Handle_t *handle);
```

【功能描述】

销毁图像旋转通道。

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

5.1.3 获取源数据共享 Buffer

【函数原型】

```
IHAL_INT32 IHal_Rot_SrcDmaBufGet(IHal_Rot_Handle_t *handle,  
                                  IMPP_BufferInfo_t *share, IHAL_INT32 index);
```

【功能描述】

获取 Rotater 通道源数据的共享 Buffer。

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
share	共享 Buffer 信息	输出
index	Buffer 编号	输入

【返回值】

成功：0

失败：非 0

【注意事项】

此 API 须在通道创建时指定源 Buffer 为内部的(IMPP_INTERNAL_BUFFER)。

5.1.4 获取输出数据共享 Buffer

【函数原型】

```
IHAL_INT32 IHal_Rot_DstDmaBufGet(IHal_Rot_Handle_t *handle,  
                                  IMPP_BufferInfo_t *share, IHAL_INT32 index);
```

【功能描述】

获取 Rotater 通道输出数据的共享 Buffer。

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
share	共享 Buffer 信息	输出
index	Buffer 编号	输入

【返回值】

成功：0

失败：非 0

【注意事项】

此 API 须在通道创建时指定输出 Buffer 为内部的 (IMPP_INTERNAL_BUFFER)。

5.1.5 设置源数据共享 Buffer

【函数原型】

```
IHAL_INT32 IHal_Rot_SetExtSrcBuffer(IHal_Rot_Handle_t *handle,  
                                      IMPP_BufferInfo_t *share, IHAL_INT32 index);
```

【功能描述】

设置 Rotater 通道源数据的共享 Buffer。

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
share	共享 Buffer 信息	输入
index	Buffer 编号	输入

【返回值】

成功：0

失败：非 0

【注意事项】

此 API 须在通道创建时指定输入 Buffer 为外部的 (IMPP_EXT_DMABUFFER / IMPP_EXT_USERBUFFER)，建议使用 dma-buf，USERBUFFER 方式，需要保证对应的物理地址的连续性。

5.1.6 设置输出数据共享 Buffer

【函数原型】

```
IHAL_INT32 IHal_Rot_SetExtDstBuffer(IHal_Rot_Handle_t *handle,  
                                     IMPP_BufferInfo_t *share, IHAL_INT32 index);
```

【功能描述】

设置 Rotater 通道输出数据的共享 Buffer。

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
share	共享 Buffer 信息	输入
index	Buffer 编号	输入

【返回值】

成功：0

失败：非 0

【注意事项】

此 API 须在通道创建时指定输出 Buffer 为外部的 (IMPP_EXT_DMABUFFER / IMPP_EXT_USERBUFFER)，建议使用 dma-buf，USERBUFFER 方式，需要保证对应的物理地址的连续性。

5.1.7 对一帧图像数据进行图像旋转处理

【函数原型】

```
IHAL_INT32 IHal_Rot_ProcessFrame(IHal_Rot_Handle_t *handle,  
                                  IMPP_FrameInfo_t *frame, IHal_Rot_ProcessType_t process_type,  
                                  IHAL_INT32 angle);
```

【功能描述】

对一帧图像数据进行图像旋转处理。

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
frame	需要处理的 src-frame 信息	输入
process_type	操作类型：旋转/翻转/镜像	输入
angle	旋转角度	输入

【返回值】

成功：0

失败：非 0

【注意事项】

若 src 是外部 buffer，则必须先设置共享才能使用，且需填充 size 和 index。

5.1.8 获取输出数据

【函数原型】

```
IHAL_INT32 IHal_Rot_GetFrame(IHal_Rot_Handle_t *handle, IMPP_FrameInfo_t *dst);
```

【功能描述】

获取输出 buffer（一帧图像旋转处理后的图像数据），获取后模块则没有该 buffer 的使用权

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
dst	输出 Buffer 信息（帧信息）	输出

【返回值】

成功：0

失败：非 0

【注意事项】

无。

5.1.9 释放输出数据

【函数原型】

```
IHAL_INT32 IHal_Rot_ReleaseFrame(IHal_Rot_Handle_t *handle, IMPP_FrameInfo_t *dst);
```

【功能描述】

释放输出 Buffer（一帧图像旋转处理后的图像数据），将 buffer 使用权给到 rotater 模块

【参数说明】

参数	描述	输入/输出
handle	Rot handle	输入
dst	帧信息	输出

【返回值】

成功：0

失败：非 0

【注意事项】

无。

5.2 Rotater 的 DMA-BUF 内存共享

示例 1，导入外部 dma-buf 作为 src:

```
IHal_Rot_ChAttr_t attr;
attr.sWidth = input_param.width;
attr.sHeight = input_param.height;
attr.srcBufType = IMPP_EXT_DMABUFFER;      /* 指定 SRC-Buffer 为外部 DMA-BUF */
/*
attr.dstBufType = IMPP_INTERNAL_BUFFER;
attr.numSrcBuf = 3;
attr.numDstBuf = 2;
attr.srcFmt = fmt;                        /* X2500 不需要设置 dstFmt */
rot_handle = IHal_Rot_CreateChan(&attr);
if(!rot_handle){
    printf("rot handle create error\r\n");
}
IMPP_BufferInfo_t share;
/* 将外部 dma-buf 设置到 Rotater */
for(int i = 0; i < attr.numSrcBuf; i++){
    IHal_GetCameraBuffers(cam_handle,i,&share);
    IHal_Rot_SetExtSrcBuffer(rot_handle,&share,i);
}
```

当 SRC 使用外部 dma-buf 时，调用 IHal_Rot_ProcessFrame 函数，需求确保 srcframe 的 dma-buf 的 fd 是已经被设置到 Rotater 模块。

示例 2，将 Rotater 的 src 导出，并共享给其它模块：

```
IHal_Rot_ChAttr_t attr;
attr.sWidth = input_param.width;
attr.sHeight = input_param.height;
attr.srcBufType = IMPP_INTERNAL_BUFFER;    /* 指定 SRC-Buffer 为外部 DMA-
BUF */
attr.dstBufType = IMPP_INTERNAL_BUFFER;
attr.numSrcBuf = 3;
attr.numDstBuf = 2;
attr.srcFmt = fmt;                        /* X2500 不需要设置 dstFmt */
rot_handle = IHal_Rot_CreateChan(&attr);
if(!rot_handle){
    printf("rot handle create error\r\n");
}
IMPP_BufferInfo_t share;
/* 将外部 dma-buf 设置到 Rotater */
for(int i = 0; i < attr.numSrcBuf; i++){
```

```
IHal_Rot_SrcDmaBufGet(rot_handle,&share,i);  
IHal_CameraSetBuffers(camera_handle,i,&share);  
}
```

5.3 使用示例

Rotater 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/img2D-example

代码功能说明：

代码文件	功能
rotate_test_x2000.c	输入一张图片进行图像旋转操作并输出保存，X2500 不适用
rotate_test_x2500.c	输入一张图片进行图像旋转操作并输出保存，仅 X2500 适用
cam_rotate_test.c	将 Camera 的视频旋转 90 度后显示在 LCD 上，仅 X2500 适用
dual_cam_rot_display_test.c	一路 Camera 旋转 90 度后显示在上半屏、一路 Camera 旋转 180 度后显示在下半屏，仅 X2500 适用。

6 Codec 模块

IMP Codec 模块主要完成视频的硬件编解码，X2000 和 X2500 平台均已支持，但是平台间硬件编解码能力上存在差异。

芯片平台	支持的编解码类型	最大性能
X2000	H264/JPEG	1080@30fps
X2500	H264/H265/JPEG (不支持解码)	4K@30fps

6.1 Codec 模块 API 介绍

6.1.1 初始化 Codec 模块

【函数原型】

```
IHAL_INT32 IHal_CodecInit(void);
```

【功能描述】

Codec 模块初始化，需要在创建 Codec 通道之前调用。

【参数说明】

无。

【返回值】

成功：0

失败：非 0

【注意事项】

无。

6.1.2 反初始化 Codec 模块

【函数原型】

```
IHAL_INT32 IHal_CodecDeInit(void);
```

【功能描述】

Codec 模块反初始化

【参数说明】

无。

【返回值】

成功：0

失败：非 0

【注意事项】

无。

6.1.3 创建编解码通道

【函数原型】

```
IHal_CodecHandle_t* IHal_CodecCreate(CODEC_TYPE type);
```

【功能描述】

创建一个编码解码能力(通道)。

【参数说明】

参数	描述	输入/输出
type	要创建的 Codec 类型: H264_ENC, H264_DEC, H265_ENC, H265_DEC, JPEG_ENC, JPEG_DEC,	输入

【返回值】

成功: Codec_Handle

失败: NULL

【注意事项】

不同的芯片，所支持的硬件编解码能力有所不同，X2000/M300/X2100 支持 H264/JPEG 的编解码；X2500 支持 H264/H265/JPEG 编码。

6.1.4 销毁编解码通道

【函数原型】

```
IHAL_INT32 IHal_CodecDestroy(IHal_CodecHandle_t *handle);
```

【功能描述】

销毁一个编码解码能力(通道)。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.5 设置编解码器参数

【函数原型】

```
IHAL_INT32 IHal_Codec_SetParams(IHal_CodecHandle_t *handle,  
                                IHal_CodecParam *param);
```

【功能描述】

设置编解码的参数，如编码图像的宽高，RC 模式，比特率等；该函数需要在编码开始前调用一次。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
param	需要设置的参数结构	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.6 获取编解码器参数

【函数原型】

```

IHAL_INT32 IHal_Codec_GetParams(IHal_CodecHandle_t *handle,
                                IHal_CodecParam *param);

```

【功能描述】

获取编解码的参数。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
param	被设置的参数结构	输出

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.7 启动编解码器

【函数原型】

```

IHAL_INT32 IHal_Codec_Start(IHal_CodecHandle_t *handle);

```

【功能描述】

Codec 开始工作。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.8 停止编解码器

【函数原型】

```

IHAL_INT32 IHal_Codec_Stop(IHal_CodecHandle_t *handle);

```

【功能描述】

Codec 停止工作。

【参数说明】

参数	描述	输入/输出
----	----	-------

handle	Codec 句柄	输入
--------	----------	----

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.9 创建源数据 Buffer

【函数原型】

```

IHAL_INT32 IHal_Codec_CreateSrcBuffers(IHal_CodecHandle_t *handle,
                                         IMPP_BUFFER_TYPE buftype, IHAL_INT32 create_num);

```

【功能描述】

创建编解码源数据缓冲 buffer。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
buftype	Buffer 类型: IMPP_INTERNAL_BUFFER: 内部 Buffer; IMPP_EXT_DMABUFFER: 外部的 dma-buf。 IMPP_EXT_USERBUFFER: 外部的用户空间 buffer,此用户空间 Buffer。	输入
create_num	需要创建 buffer 的个数, 最大值 3 个超过最大值, 则默认为 3。	输入

【返回值】

成功: 创建 buffer 的实际个数

失败: -1

【注意事项】

通常建议实际应用时, 编码器采用外部 dma-buf, 内部 buffer 通常用于测试。

6.1.10 设置源数据 Buffer

【函数原型】

```

IHAL_INT32 IHal_Codec_SetSrcBuffer(IHal_CodecHandle_t *handle,
                                     int index, IMPP_BufferInfo_t *sharebuf);

```

【功能描述】

设置 Codec 源数据 buffer, 该函数用导入外部的 buffer(如 camera 的数据 buffer)作为编解码的源数据, 在编码开始工作之前进行设置; 通常编解码器更加的适用 dma-buf 这类能够保证物理上连续的地址空间。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
index	Buffer 序号	输入
sharebuf	Buf 信息	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.11 获取源数据共享 Buffer

【函数原型】

```

IHAL_INT32 IHal_Codec_GetSrcBuffer(IHal_CodecHandle_t *handle, int index,
                                     IMPP_BufferInfo_t *sharebuf);

```

【功能描述】

当编解码器源数据使用内部 buffer 时，可以调用此函数将 buffer 导出，用于和其他模块的共享。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
index	Buffer 序号	输入
Sharebuf	Buf 信息	输出

【返回值】

成功: 0

失败: 非 0

【注意事项】

通常建议编码器使用外部 dma-buf，即作为 dma-buf 的消费者。

6.1.12 创建输出数据 Buffer

【函数原型】

```

IHAL_INT32 IHal_Codec_CreateDstBuffer(IHal_CodecHandle_t *handle,
                                       IMPP_BUFFER_TYPE buftype, IHAL_INT32 create_num);

```

【功能描述】

创建编码器的输出数据 buffer。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
buftype	Buffer 类型: IMPP_INTERNAL_BUFFER: 内部 Buffer; IMPP_EXT_DMABUFFER: 外部的 dma-buf。 IMPP_EXT_USERBUFFER: 外部的用户空间 buffer,此用户空间 Buffer。	输入
create_num	需要创建 buffer 的个数，最大值 3 个，超过最大值，则默认为 3。	输入

【返回值】

成功: 实际创建 buffer 的个数

失败: -1

【注意事项】

编码器由于码流数据较小，一般不需要进行内存共享，因此编码时只支持创建内部 **buffer**。

6.1.13 设置输出数据 Buffer

【函数原型】

```
IHAL_INT32 IHal_Codec_SetDstBuffer(IHal_CodecHandle_t *handle,  
                                     int index, IMPP_BufferInfo_t *sharebuf);
```

【功能描述】

设置 Codec 输出数据 **buffer**，该函数用导入外部的 **buffer**（如显示模块）作为编解码的输出缓冲，在编解码器开始工作之前进行设置；通常编解码器更加的适用 **dma-buf** 这类能够保证物理上连续的地址空间。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
index	Buffer 序号	输入
sharebuf	Buf 信息	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

此 API 不适用编码器。

6.1.14 获取输出共享 Buffer

【函数原型】

```
IHAL_INT32 IHal_Codec_GetDstBuffer(IHal_CodecHandle_t *handle,  
                                     int index, IMPP_BufferInfo_t *sharebuf);
```

【功能描述】

获取编解码的输出 **buffer**，用于和其他模块的共享；如在解码时，可以将解码的输出 **buffer** 用于 DPU 显示。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
index	Buffer 序号	输入
sharebuf	Buffer 信息	输出

【返回值】

成功: 0

失败: 非 0

【注意事项】

此 API 不适用编码器。

6.1.15 放入源数据 Buffer

【函数原型】

```
IHAL_INT32 IHaI_Codec_QueueSrcBuffer(IHaI_CodecHandle_t *handle,  
                                       IMPP_BufferInfo_t *buf);
```

【功能描述】

将 Codec 的源数据 buffer 放入队列，在源数据填充完毕后需要通过此函数将 buffer 给到编解码器。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
buf	Buf 信息	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

调用此 API 必须先等待 buffer 可用。

6.1.16 等待源数据 Buffer

【函数原型】

```
IHAL_INT32 IHaI_Codec_WaitSrcAvailable(IHaI_CodecHandle_t* handle,  
                                         IHAL_INT32 wait_type);
```

【功能描述】

等待 Codec 的源数据 buffer 可用。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
wait_type	等待类型，非阻塞等待和阻塞等待。 IMPP_WAIT_FOREVER IMPP_NO_WAIT	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.17 获取源数据 Buffer

【函数原型】

```
IHAL_INT32 IHaI_Codec_DequeueSrcBuffer(IHaI_CodecHandle_t *handle,  
                                         IMPP_BufferInfo_t *buf);
```

【功能描述】

将源数据 Buffer 出队列，当 Codec 使用完毕后，需要调用此函数将 buffer 出队列并继续填充

源数据。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
buf	Codecbuffer 信息	输出

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.18 放入输出数据 Buffer

【函数原型】

```

IHAL_INT32 IHal_Codec_QueueDstBuffer(IHal_CodecHandle_t *handle,
                                       IHAL_CodecStreamInfo_t *buf);

```

【功能描述】

将 Codec 的输出缓冲 buffer 放入队列，当应用使用完输出 buffer 后，需要调用此接口归还 buffer 到 Codec。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
buf	编解码器输出流信息	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.19 等待输出数据 Buffer

【函数原型】

```

IHAL_INT32 IHal_Codec_WaitDstAvailable(IHal_CodecHandle_t* handle,
                                         IHAL_INT32 wait_type);

```

【功能描述】

等待编解码器输出 buffer 可用，例如在编码时，就是等待可用的码流输出。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
wait_type	等待类型，非阻塞等待和阻塞等待。 IMPP_WAIT_FOREVER IMPP_NO_WAIT	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.20 获取输出数据 Buffer**【函数原型】**

```
IHAL_INT32 IHaI_Codec_DequeueDstBuffer(IHaI_CodecHandle_t *handle,  
                                         IHaI_CodecStreamInfo_t *buf);
```

【功能描述】

将 Codec 的输出数据 buffer 出队列，如当编码完成后，应用可以通过此接口获取到一帧码流数据。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
buf	Codecbuffer 信息	输出

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.1.21 设置控制参数**【函数原型】**

```
IHAL_INT32 IHaI_Codec_Control(IHaI_CodecHandle_t *handle,  
                               IHaI_CodecControlParam_t *param);
```

【功能描述】

设置编解码器的控制参数。

【参数说明】

参数	描述	输入/输出
handle	Codec 句柄	输入
param	编解码器控制参数	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

6.2 使用示例

Codec 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/codec-example

代码功能说明：

代码文件	功能
camera_h264enc_test_v1.c	进行单路摄像头数据（NV12 格式）的 h264 编码，并保存码流数据
camera_h265enc_test_v1.c	进行单路摄像头数据（NV12 格式）的 h265 编码，并保存码流数据，仅 X2500 支持
file_jpegenc_x2000_test.c	对一张 NV12 格式的图片进行 JPEG 编码，适用于 x2000
file_jpegenc_x2500_test.c	对一张 NV12 格式的图片进行 JPEG 编码，仅适用 X2500
camera_jpegenc_test.c	单路摄像头数据的 jpeg 编码，并保存 jpg 图片
dual_camera_h264enc_test_v1.c	双路视频的 h264 编码
dual_camera_h264enc_h265enc.c	一路 h264 编码，一路 h265 编码；该代码仅适用于 X2500
h264dec_test.c	将一个 H264 编码的码流文件进行解码并通过 LCD 进行显示
jpegdec_test.c	将一张 JPEG 文件解码为 NV12 格式并输出保存

输入数据格式说明：

输出格式 芯片平台	JPEG	H264 H265(X2500 支持)
X2000	NV12 / NV21	YUV420P / NV12 / NV21
X2500	NV12/NV21	NV12

7 Audio 模块

IMPP Audio 模块支持音频数据的采集、播放、编码和解码，采集时音频处理方面支持 AEC、AGC、NS、HPF，支持音频重采样。

7.1 Audio 模块 API 介绍

7.1.1 创建音频输入通道

【函数原型】

```
IHal_AudioHandle_t* IHal_AI_ChanCreate(IHal_AI_Attr_t *attr);
```

【功能描述】

创建一个音频输入通道。

【参数说明】

参数	描述	输入/输出
Attr	录音通道属性参数	输入

【返回值】

成功: Audio_Handle;

失败: IHAL_RNULL

【注意事项】

无。

7.1.2 销毁音频输入通道

【函数原型】

```
IHAL_INT32 IHal_AI_ChanDestroy(IHal_AudioHandle_t *handle);
```

【功能描述】

销毁音频输入通道。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

7.1.3 启动音频输入通道

【函数原型】

```
IHAL_INT32 IHal_AI_ChanStart(IHal_AudioHandle_t *handle);
```

【功能描述】

音频输入通道开始工作。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功： 0

失败：非 0

【注意事项】

无。

7.1.4 停止音频输入通道

【函数原型】

```
IHAL_INT32 IHal_AI_ChangeStop(IHal_AudioHandle_t *handle);
```

【功能描述】

音频输入通道停止工作。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功： 0

失败：非 0

7.1.5 获取一帧音频输入数据

【函数原型】

```
IHAL_INT32 IHal_AI_GetBuffer(IHal_AudioHandle_t *handle,  
                             IHal_AudioBuffer_t *buf, IHAL_INT32 wait);
```

【功能描述】

获取一帧 PCM 数据。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
buf	Buffer 信息	输出
wait	等待类型 (IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER)	输入

【返回值】

成功： 0

失败：非 0

7.1.6 释放音频输入数据

【函数原型】

```
IHAL_INT32 IHal_AI_ReleaseBuffer(IHal_AudioHandle_t *handle, IHal_AudioBuffer_t *buf);
```

【功能描述】

释放音频数据 buffer。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

buf	Buffer 信息	输入
-----	-----------	----

【返回值】

成功： 0

失败：非 0

7.1.7 设置录音音量

【函数原型】

IHAL_INT32 IHal_AI_SetVolume(IHal_AudioHandle_t *handle, IHAL_INT32 value);

【功能描述】

设置录音音量。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
value	音量值 (0-100)	输入

【返回值】

成功： 0

失败：非 0

7.1.8 获取录音音量

【函数原型】

IHAL_INT32 IHal_AI_GetVolume(IHal_AudioHandle_t *handle, IHAL_INT32 *value);

【功能描述】

获取录音音量。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
value	音量值 (0-100)	输出

【返回值】

成功： 0

失败：非 0

7.1.9 获取录音数字增益值

【函数原型】

IHAL_INT32 IHal_AI_GetDigitalGain(IHal_AudioHandle_t *handle, IHAL_INT32 *val);

【功能描述】

获取录音数字增益值。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
val	增益值	输出

【返回值】

成功： 0

失败：非 0

7.1.10 使能回声消除

【函数原型】

```
IHAL_INT32 IHal_AI_EnableAec(IHal_AudioHandle_t *handle);
```

【功能描述】

使能回声消除。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功： 0

失败：非 0

7.1.11 关闭回声消除

【函数原型】

```
IHAL_INT32 IHal_AI_DisableAec(IHal_AudioHandle_t *handle);
```

【功能描述】

关闭回声消除。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功： 0

失败：非 0

7.1.12 单声道选择

【函数原型】

```
IHAL_INT32 IHal_AI_SingleChannelSet(IHal_AudioHandle_t *handle,  
                                     IHal_Audio_MonoType_t mono);
```

【功能描述】

单声道录音时，指定哪个录音声道。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
mono	声道选择	输入

【返回值】

成功： 0

失败：非 0

7.1.13 使能自动增益

【函数原型】

IHAL_INT32 IHal_AI_EnableAgc(IHal_AudioHandle_t *handle, IHal_AudioAgcConfig_t *config);

【功能描述】

使能录音自动增益。

【参数说明】

参数	描述	输入/输出
Handle	Audio_handle	输入
Config	增益配置	输入

【返回值】

成功： 0

失败：非 0

7.1.14 关闭自动增益

【函数原型】

IHAL_INT32 IHal_AI_DisableAgc(IHal_AudioHandle_t *handle);

【功能描述】

关闭录音增益。

【参数说明】

参数	描述	输入/输出
Handle	Audio_handle	输入

【返回值】

成功： 0

失败：非 0

7.1.15 使能降噪

【函数原型】

IHAL_INT32 IHal_AI_EnableNs(IHal_AudioHandle_t *handle, int mode);

【功能描述】

使能录音降噪。

【参数说明】

参数	描述	输入/输出
Handle	Audio_handle	输入
Mode	降噪模式	输入

【返回值】

成功： 0

失败：非 0

7.1.16 关闭降噪

【函数原型】

IHAL_INT32 IHal_AI_DisableNs(IHal_AudioHandle_t *handle);

【功能描述】

关闭录音降噪。

【参数说明】

参数	描述	输入/输出
Handle	Audio_handle	输入

【返回值】

成功： 0

失败：非 0

7.1.17 使能高通滤波

【函数原型】

IHAL_INT32 IHal_AI_EnableHp(IHal_AudioHandle_t *handle, unsigned int cutoff_Freq);

【功能描述】

使能录音高通滤波。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
cutoff_Freq	截止频率	输入

【返回值】

成功： 0

失败：非 0

7.1.18 关闭高通滤波

【函数原型】

IHAL_INT32 IHal_AI_DisableHp(IHal_AudioHandle_t *handle);

【功能描述】

关闭录音高通滤波。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
cutoff_Freq	截止频率	输入

【返回值】

成功： 0

失败：非 0

7.1.19 创建音频输出通道

【函数原型】

IHal_AudioHandle_t* IHal_AO_ChangeCreate(IHal_AO_Attr_t *attr);

【功能描述】

创建音频输出通道。

【参数说明】

参数	描述	输入/输出
attr	音频输出通道属性	输入

【返回值】

成功： AudioHandle

失败: IHAL_RNULL

7.1.20 销毁音频输出通道

【函数原型】

```
IHAL_INT32 IHal_AO_ChanDestroy(IHal_AudioHandle_t *handle);
```

【功能描述】

销毁音频输出通道。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功: 0

失败: 非 0

7.1.21 启动音频输出通道

【函数原型】

```
IHAL_INT32 IHal_AO_ChanStart(IHal_AudioHandle_t* handle);
```

【功能描述】

音频输出通道开始工作。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功: 0

失败: 非 0

7.1.22 停止音频输出通道

【函数原型】

```
IHAL_INT32 IHal_AO_ChanStop(IHal_AudioHandle_t* handle);
```

【功能描述】

音频输出通道停止工作。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功: 0

失败: 非 0

7.1.23 写入音频输出数据

【函数原型】

```
IHAL_INT32 IHal_AO_ChanWriteData(IHal_AudioHandle_t *handle,  
                                   IHal_AudioFrm_t *frame, IHAL_INT32 waitblock);
```

【功能描述】

音频输出通道写入一帧数据。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
frame	写入的数据帧	输入
waitblock	等待类型（IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER）	输入

【返回值】

成功：0

失败：非 0

7.1.24 重启音频输出通道

【函数原型】

```
IHAL_INT32 IHal_AO_ChanReStart(IHal_AudioHandle_t* handle);
```

【功能描述】

立即中断当前的播放状态，并清空缓冲区，等待新的数据写入。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功：0

失败：非 0

7.1.25 暂停音频播放

【函数原型】

```
IHAL_INT32 IHal_AO_Pause(IHal_AudioHandle_t *handle);
```

【功能描述】

暂停播放。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功：0

失败：非 0

7.1.26 恢复音频播放

【函数原型】

```
IHAL_INT32 IHal_AO_Resume(IHal_AudioHandle_t *handle);
```

【功能描述】

恢复音频播放。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功: 0

失败: 非 0

7.1.27 刷新音频输出缓冲区

【函数原型】

IHAL_INT32 IHal_AO_BufferFlush(IHal_AudioHandle_t *handle);

【功能描述】

等待缓冲区中所有数据被发送到硬件，防止缺失数据，可在停止播放前调用。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入

【返回值】

成功: 0

失败: 非 0

7.1.28 设置音频输出静音状态

【函数原型】

IHAL_INT32 IHal_AO_SetMute(IHal_AudioHandle_t *handle, IHAL_INT32 mute);

【功能描述】

设置静音状态。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
mute	静音状态 (0, 1)	输入

【返回值】

成功: 0

失败: 非 0

7.1.29 获取音频输出静音状态

【函数原型】

IHAL_INT32 IHal_AO_GetMuteStatus(IHal_AudioHandle_t *handle, IHAL_INT32 *mute);

【功能描述】

获取静音状态。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
mute	静音状态	输出

【返回值】

成功: 0

失败：非 0

7.1.30 设置音频输出音量

【函数原型】

```
IHAL_INT32 IHal_AO_SetVolume(IHal_AudioHandle_t *handle, IHAL_INT32 value);
```

【功能描述】

设置音频播放音量。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
value	音量值 (0-100)	输入

【返回值】

成功：0

失败：非 0

7.1.31 获取音频输出音量

【函数原型】

```
IHAL_INT32 IHal_AO_GetVolume(IHal_AudioHandle_t *handle, IHAL_INT32 *value);
```

【功能描述】

获取音频播放音量。

【参数说明】

参数	描述	输入/输出
handle	Audio_handle	输入
value	音量值 (0-100)	输出

【返回值】

成功：0

失败：非 0

7.1.32 创建音频编码通道

【函数原型】

```
IHal_AudioEnc_Handle_t* IHal_AudioEnc_CreateChn(IHal_AudioEnc_Attr_t *attr);
```

【功能描述】

创建音频编码通道，目前支持的音频编码类型：ADPCM、G711A、G711U、G726、AAC。

【参数说明】

参数	描述	输入/输出
attr	音频编码属性	输入

【返回值】

成功：handle

失败：NULL

7.1.33 销毁音频编码通道

【函数原型】

```
IHAl_INT32 IHal_AudioEnc_DestroyChn(IHal_AudioEnc_Handle_t *handle);
```

【功能描述】

销毁音频编码通道。

【参数说明】

参数	描述	输入/输出
handle	Audio encoder handle	输入

【返回值】

成功：0

失败：非 0

7.1.34 发送原始数据

【函数原型】

```
IHAl_INT32 IHal_AudioEnc_PutFrame(IHal_AudioEnc_Handle_t *handle,  
                                   IHal_AudioStream_t *frame, IHAl_INT32 block);
```

【功能描述】

写入原始数据到编码通道。

【参数说明】

参数	描述	输入/输出
handle	Audio encoder handle	输入
frame	音频原始数据信息	输入
Block	阻塞状态（IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER）	输入

【返回值】

成功：0

失败：非 0

7.1.35 获取编码数据

【函数原型】

```
IHAl_INT32 IHal_AudioEnc_GetStream(IHal_AudioEnc_Handle_t *handle,  
                                    IHal_AudioStream_t *stream, IHAl_INT32 block);
```

【功能描述】

从编码器获取编码后的数据。

【参数说明】

参数	描述	输入/输出
handle	Audio encoder handle	输入
stream	编码数据信息	输出
block	阻塞状态（IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER）	输入

【返回值】

成功：0

失败：非 0

7.1.36 释放编码数据

【函数原型】

```
IHAL_INT32 IHal_AudioEnc_ReleaseStream(IHal_AudioEnc_Handle_t *handle,  
                                         IHal_AudioStream_t *stream);
```

【功能描述】

释放一帧编码后的音频输入数据。

【参数说明】

参数	描述	输入/输出
handle	Audio encoder handle	输入
stream	编码数据 buffer 信息	输入

【返回值】

成功：0

失败：非 0

7.1.37 创建音频解码通道

【函数原型】

```
IHal_AudioDec_Handle_t* IHal_AudioDec_CreateChn(IHal_AudioDec_Attr_t *attr);
```

【功能描述】

创建音频解码通道，目前支持 ADPCM、G711A、G711U、G726、AAC。

【参数说明】

参数	描述	输入/输出
attr	解码通道属性	输入

【返回值】

成功：handle

失败：NULL

7.1.38 销毁音频解码通道

【函数原型】

```
IHAL_INT32 IHal_AudioDec_DestroyChn(IHal_AudioDec_Handle_t *handle);
```

【功能描述】

销毁一个音频解码器通道。

【参数说明】

参数	描述	输入/输出
handle	Audio decoder handle	输入

【返回值】

成功：0

失败：非 0

7.1.39 发送压缩数据

【函数原型】

```
IHAL_INT32 IHal_AudioDec_PutStream(IHal_AudioDec_Handle_t *handle,
```

IHal_AudioStream_t *stream, IHAL_INT32 block);

【功能描述】

发送音频压缩码流数据到音频解码器。

【参数说明】

参数	描述	输入/输出
handle	Audio decoder handle	输入
stream	音频压缩数据信息	输入
block	阻塞状态（IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER）	输入

【返回值】

成功：0

失败：非 0

7.1.40 获取解码数据

【函数原型】

IHAI_INT32 IHal_AudioDec_GetFrame(IHal_AudioEnc_Handle_t *handle,
IHal_AudioStream_t *frame, IHAL_INT32 block);

【功能描述】

从音频解码器获取一帧解码后的音频数据。

【参数说明】

参数	描述	输入/输出
handle	Audio decoder handle	输入
frame	PCM 数据信息	输出
block	阻塞状态（IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER）	输入

【返回值】

成功：0

失败：非 0

7.1.41 释放解码数据

【函数原型】

IHAI_INT32 IHal_AudioDec_ReleaseFrame(IHal_AudioEnc_Handle_t *handle,
IHal_AudioStream_t *frame);

【功能描述】

释放一帧解码后的音频数据。

【参数说明】

参数	描述	输入/输出
handle	Audio decoder handle	输入
frame	PCM 数据 buffer 信息	输入

【返回值】

成功：0

失败：非 0

7.1.42 重采样初始化

【函数原型】

```
IHal_Resampler_Handle_t* IHal_ResamplerInit(IHal_ResamplerAttr_t *attr);
```

【功能描述】

初始化重采样通道。

【参数说明】

参数	描述	输入/输出
attr	重采样属性	输入

【返回值】

成功: handle

失败: NULL

7.1.43 重采样反初始化

【函数原型】

```
IHAL_INT32 IHal_ResamplerDelnit(IHal_Resampler_Handle_t *handle);
```

【功能描述】

反初始化重采样通道。

【参数说明】

参数	描述	输入/输出
handle	Resampler handle	输入

【返回值】

成功: 0

失败: 非 0

7.1.44 重采样处理

【函数原型】

```
IHAL_INT32 IHal_ResamplerProcess(IHal_Resampler_Handle_t* handle, IHAL_INT16* indata,  
IHAL_INT16** outdata, IHAL_INT32 insize, IHAL_INT32* outsize);
```

【功能描述】

重采样处理。

【参数说明】

参数	描述	输入/输出
handle	Resampler handle	输入
indata	输入数据	输入
outdata	输出数据	输出
insize	输入数据大小	输入
outsize	输出数据大小	输出

【返回值】

成功: 0

失败: 非 0

7.1.45 设置重采样率

【函数原型】

```

IHAL_INT32 IHal_ResamplerSetRate(IHal_Resampler_Handle_t* handle,
                                   IHAL_UINT32 in_samplerate, IHAL_UINT32 out_samplerate);

```

【功能描述】

设置重采样率，用于修改通道参数。

【参数说明】

参数	描述	输入/输出
handle	Resampler handle	输入
in_samplerate	输入数据采样率	输入
out_samplerate	输出数据采样率	输入

【返回值】

成功：0

失败：非 0

7.1.46 获取重采样率

【函数原型】

```

IHAL_INT32 IHal_ResamplerGetRate(IHal_Resampler_Handle_t* handle,
                                   IHAL_UINT32 *in_samplerate, IHAL_UINT32 *out_samplerate);

```

【功能描述】

获取通道采样率参数。

【参数说明】

参数	描述	输入/输出
handle	Resampler handle	输入
in_samplerate	输入数据采样率	输出
out_samplerate	输出数据采样率	输出

【返回值】

成功：0

失败：非 0

7.1.47 设置重采样质量

【函数原型】

```

IHAL_INT32 IHal_ResamplerSetQuality(IHal_Resampler_Handle_t* handle,
                                      IHAL_UINT32 target_quality);

```

【功能描述】

设置重采样质量参数，用于参数修改。

【参数说明】

参数	描述	输入/输出
handle	Resampler handle	输入
target_quality	质量参数 (0-10)	输入

【返回值】

成功：0

失败：非 0

7.2 使用示例

Audio 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/audio_test

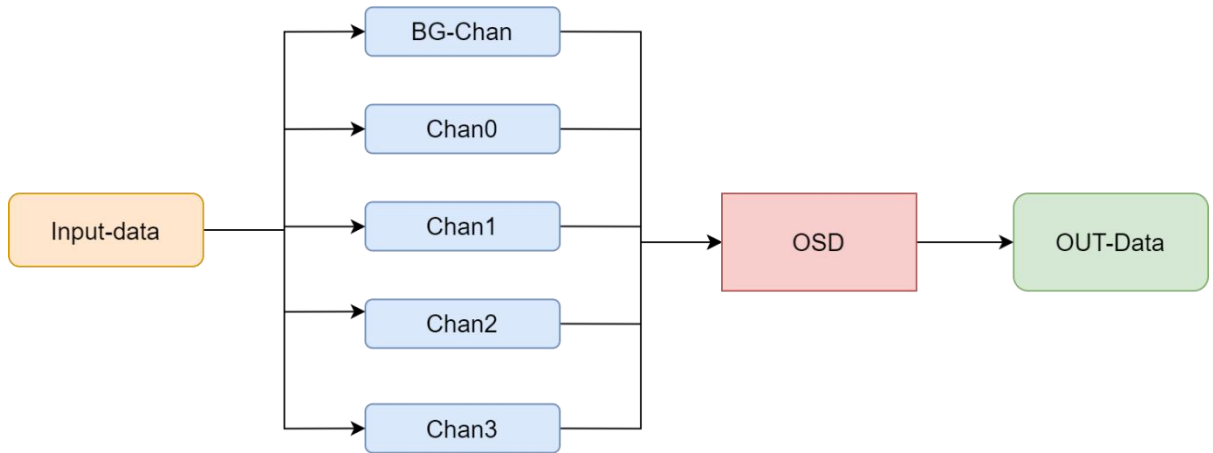
代码功能说明：

代码文件	功能
ai_test.c	录音，并保存为 pcm 文件
ao_test.c	播放一个 pcm 音频文件
ai_aenc_test.c	对输入的音频进行编码并保存为音频文件
ao_adec_test.c	对音频文件进行解码并播放，同时将解码后的音频数据保存为 PCM 音频文件
aec_test.c	回声消除测试（目前仅 X2500 支持）
resampler_test.c	输入一个音频文件，对其进行重采样并输出保存

8 IPU 模块

IMPP IPU 模块, 支持 OSD 和 CSC 两大功能, 该模块仅支持 X2500 芯片。

OSD 功能示意:



8.1 IPU 模块 API 介绍

8.1.1 创建格式转换通道

【函数原型】

IHal_CSC_Handle_t* IHal_CSC_Change(IHal_CSC_ChangeAttr_t *attr);

【功能描述】

创建格式转换通道

【参数说明】

参数	描述	输入/输出
attr	格式转换通道属性	输入

【返回值】

成功: handle

失败: NULL

【注意事项】

无。

8.1.2 销毁格式转换通道

【函数原型】

HAL_INT32 IHal_CSC_DestroyChan(IHal_CSC_Handle_t *handle);

【功能描述】

销毁格式转换通道

【参数说明】

参数	描述	输入/输出
handle	CSC handle	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

8.1.3 获取源数据共享 Buffer

【函数原型】

```

IHAL_INT32 IHal_CSC_GetSrcBuf(IHal_CSC_Handle_t *handle,
                               IMPP_BufferInfo_t *buf, IHAL_INT32 index);

```

【功能描述】

获取源数据的 dma-buf，用于内存共享

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
buf	共享 dma-buf 信息	输出
index	buf-index	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

此 API 使用的前提是: 创建 CSC 通道时, 源数据不使用外部 dma-buf。

8.1.4 获取输出数据共享 Buffer

【函数原型】

```

IHAL_INT32 IHal_CSC_GetDstBuf(IHal_CSC_Handle_t *handle,
                               IMPP_BufferInfo_t *buf, IHAL_INT32 index);

```

【功能描述】

获取输出数据的 dma-buf，用于内存共享

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
buf	共享 dma-buf 信息	输出
index	buf-index	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

此 API 使用的前提是: 创建 CSC 通道时, 输出数据不使用外部 dma-buf。

8.1.5 设置源数据共享 Buffer

【函数原型】

```

IHAL_INT32 IHal_CSC_SetSrcExtBuf(IHal_CSC_Handle_t *handle,
                                   IMPP_BufferInfo_t *buf, IHAL_INT32 index);

```

【功能描述】

设置格式转换通道源数据共享 buffer，用于共享缓冲区。

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
buf	共享 dma-buf 信息	输入
index	buf-index	输入

【返回值】

成功：0

失败：非 0

【注意事项】

此 API 使用的前提是：创建 CSC 通道时，源数据必须使用外部 dma-buf；参数 buf 的 fd 和 size 成员必须是有效的。

8.1.6 设置输出数据共享 Buffer

【函数原型】

```
IHAL_INT32 IHal_CSC_SetDstExtBuf(IHal_CSC_Handle_t *handle,  
IMPP_BufferInfo_t *buf, IHAL_INT32 index);
```

【功能描述】

设置格式转换通道输出数据共享 buffer，用于共享缓冲区。

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
buf	共享 dma-buf 信息	输入
index	buf-index	输入

【返回值】

成功：0

失败：非 0

【注意事项】

此 API 使用的前提是：创建 CSC 通道时，输出数据必须使用外部 dma-buf；参数 buf 的 fd 和 size 成员必须是有效的。

8.1.7 导入 dma-buf

【函数原型】

```
IHAL_INT32 IHal_CSC_ImportDmaBuf(IHal_CSC_Handle_t *handle,  
IMPP_BufferInfo_t *buffer);
```

【功能描述】

导入 dma-buf，可以获取其物理地址。

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
buffer	缓冲区信息	输出

【返回值】

成功：0

失败：非 0

【注意事项】

参数 **buffer** 的 **fd** 和 **size** 成员必须是有效的。API 调用成功后，会填充参数 **buffer** 的物理地址字段。

8.1.8 格式转换处理

【函数原型】

```

IHAL_INT32 IHaI_CSC_FrameProcess(IHaI_CSC_Handle_t *handle,
                                  IMPP_FrameInfo_t *frame);

```

【功能描述】

对输入的一帧图像数据进行格式转换处理。

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
frame	源 frame 信息	输入

【返回值】

成功：0

失败：非 0

【注意事项】

如果 **frame** 是外部模块共享的 **dma-buf**，那么必须调用相关 API 进行设置共享 **buffer**。

8.1.9 获取格式转换输出数据

【函数原型】

```

IHAL_INT32 IHaI_CSC_GetDstFrame(IHaI_CSC_Handle_t *handle,
                                  IMPP_FrameInfo_t *frame, IHAL_INT32 wait);

```

【功能描述】

获取格式转换后的数据。

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
frame	输出 frame 信息	输出
wait	等 待 类 型 （ IMPP_NO_WAIT 或 IMPP_WAIT_FOREVER ）	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

8.1.10 释放格式转换输出数据

【函数原型】

```

IHAL_INT32 IHaI_CSC_ReleaseDstFrame(IHaI_CSC_Handle_t *handle,
                                     IMPP_FrameInfo_t *frame);

```

【功能描述】

释放输出 buffer，将使用权给到 CSC

【参数说明】

参数	描述	输入/输出
handle	CSC-handle	输入
frame	要释放 frame 信息	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

在输出数据使用完之后，需要调用此 API。

8.1.11 刷新 CSC 通道 Buffer 的 Cache

【函数原型】

```

IHAL_INT32 IHaI_CSC_FlushCache(IHaI_CSC_Handle_t *handle,
                                IPU_DmaBuf_SyncInfo_t *info);

```

【功能描述】

刷新 CSC dma-buf 的 Cache 缓存。

【参数说明】

参数	描述	输入/输出
handle	CSC-Handle	输入
info	需要刷 cache 的 dma-buf 信息	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

当 buffer 需要被用户空间使用时，需要调用此 API，来保证 cache 的一致。

8.1.12 创建 OSD 通道

【函数原型】

```

IHaI_OSD_Handle_t* IHaI_OSD_ChanCreate(IHaI_OSD_ChAttr_t *attr);

```

【功能描述】

创建 OSD 通道。

【参数说明】

参数	描述	输入/输出
attr	通道属性	输入

【返回值】

成功: OSD handle

失败: NULL

【注意事项】

无。

8.1.13 销毁 OSD 通道

【函数原型】

```
IHAL_INT32 IHal_OSD_DestroyChan(IHal_OSD_Handle_t *handle);
```

【功能描述】

销毁 OSD 通道。

【参数说明】

参数	描述	输入/输出
handle	OSD-Handle	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

8.1.14 创建 OSD 通道的数据 Buffer

【函数原型】

```
IHAL_INT32 IHal_OSD_CHx_BufCreate(IHal_OSD_Handle_t *handle,  
IMPP_BufferInfo_t *buffer, IHAL_UINT32 size);
```

【功能描述】

为 OSD 相关通道创建 buffer，默认导出了 dma-buf，同时可得到物理地址和虚拟地址。

【参数说明】

参数	描述	输入/输出
handle	OSD-Handle	输入
buffer	Buffer 信息	输出
size	创建 Buffer 的大小	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

创建 buffer 时，内部是按照页对齐的，因此实际大小可能会大于 size。

8.1.15 释放 OSD 通道数据 Buffer

【函数原型】

```
IHAL_INT32 IHal_OSD_CHx_BufFree(IHal_OSD_Handle_t *handle,  
IMPP_BufferInfo_t *buffer);
```

【功能描述】

释放 OSD 为 OSD 通道创建的 buffer。

【参数说明】

参数	描述	输入/输出
handle	OSD-Handle	输入
buffer	Buffer 信息	输入

【返回值】

成功：0

失败：非 0

【注意事项】

通道使用完，必须进行释放，否则会内存泄漏。

8.1.16 导入 dma-buf

【函数原型】

```

IHAl_INT32 IHal_OSD_ImportDmaBuf(IHal_OSD_Handle_t *handle,
                                  IMPP_BufferInfo_t *buffer);

```

【功能描述】

导入 dma-buf，通过此 API 可获得外部共享 dma-buf 的物理地址，因为 OSD 在进行处理时参数需要直接填写物理地址。

【参数说明】

参数	描述	输入/输出
handle	OSD-Handle	输入
buffer	Buffer 信息	输出

【返回值】

成功：0

失败：非 0

【注意事项】

物理地址的获取，API 调用成功后，会填充参数 buffer 的物理地址字段。

8.1.17 OSD 处理

【函数原型】

```

IHAl_INT32 IHal_OSD_ProcessFrame(IHal_OSD_Handle_t *handle,
                                  IHal_OSD_FrameDesc_t *framedesc, IMPP_FrameInfo_t *out);

```

【功能描述】

进行一次 OSD 图像叠加操作。

【参数说明】

参数	描述	输入/输出
handle	OSD-Handle	输入
framedesc	OSD 处理帧的描述信息	输入
out	OSD 的输出	输出

【返回值】

成功：0

失败：非 0

【注意事项】

关于 OSD 输出，最终是输出到背景通道的，因此若要获取到输出的物理地址，以及 dma-buf

信息，可以通过 out 参数的 index 字段确定输出 buffer，对应的背景通道 buffer 的编号。

8.1.18 刷新 OSD 通道 Buffer 的 Cache

【函数原型】

```

IHAl_INT32 IHAl_OSD_FlushCache(IHAl_OSD_Handle_t *handle,
                                IPU_DmaBuf_SyncInfo_t *info);

```

【功能描述】

刷 OSD dma-buf 的 cache 缓存。

【参数说明】

参数	描述	输入/输出
handle	OSD-Handle	输入
info	需要刷 cache 的 dma-buf 信息	输入

【返回值】

成功：0

失败：非 0

【注意事项】

当 buffer 需要被用户空间使用时，需要调用此 API，来保证 cache 的一致。

8.2 使用示例

IPU 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/img2D-example

代码功能说明：

代码文件	功能
csc_test.c	对一张 NV12 格式的图片进行颜色空间转换并输出保存
osd_test.c	将两张（或四张）图片叠加在一张 NV12 格式的背景图片上
camera_csc_test.c	将 Camera 输出的一帧数据进行颜色空间转换并输出保存
camera_osd_enc_display.c	将背景图片与 Camera 视频进行叠加处理在 LCD 上显示并进行 h264 编码保存输出码流视频
camera_pic_osd_display_test.c	将 Camera 输出的数据经裁剪缩放后与 NV12 图片进行叠加，叠加后在 LCD 上进行显示
camera_pic_rotater_csc_dpu_osd_test.c	将 Camera 输出的图像数据旋转 90 度之后再转换成 ARGB8888 格式后与一张 ARGB8888 格式图片进行叠加显示

9 DrawBox 模块

IMPP DrawBox 模块的主要功能是在图像中画框。所画框数据以及图像相关数据从 CPU 通过 AHB 控制总线传入 drawbox 模块进行处理画框，并写入 AXI 总线，示意图如下：



9.1 DrawBox 模块 API 介绍

9.1.1 初始化 DrawBox 模块

【函数原型】

```
IHAL_INT32 IHal_DrawBox_Init(IHAL_INT32 ch);
```

【功能描述】

初始化 DrawBox 模块。

【参数说明】

参数	描述	输入/输出
ch	通道号 (0 - 1)	输入

【返回值】

成功: 0

失败: 非 0

【注意事项】

无。

9.1.2 去初始化 DrawBox 模块

【函数原型】

```
IHAL_INT32 IHal_DrawBox_DeInit(IHAL_INT32 ch);
```

【功能描述】

去初始化 DrawBox 模块。

【参数说明】

参数	描述	输入/输出
ch	通道号 (0 - 1)	输入

【返回值】

成功: 0

失败：非 0

【注意事项】

无。

9.1.3 导入 dma-buf，获取其物理地址

【函数原型】

```
IHAL_UINT32 IHal_DrawBox_ImportDmaBuf(IHAL_INT32 ch, IMPP_BufferInfo_t *buf);
```

【功能描述】

导入 dma-buf。调用这个 API 可以获得 dma-buf 的 paddr，因为 DrawBox 在进行处理时参数需要直接填写物理地址。

【参数说明】

参数	描述	输入/输出
ch	通道号 (0 - 1)	输入
buf	Buffer 信息	输入

【返回值】

成功：物理地址

失败：0

【注意事项】

无。

9.1.4 DrawBox 模块进行画框处理

【函数原型】

```
IHAL_INT32 IHal_DrawBox_Process(IHAL_INT32 ch, IHal_DrawBoxInfo_t *boxinfo);
```

【功能描述】

DrawBox 模块进行画框处理。

【参数说明】

参数	描述	输入/输出
ch	通道号 (0 - 1)	输入
boxinfo	画框信息	输入

【返回值】

成功：0

失败：非 0

【注意事项】

无。

9.2 使用示例

IPU 测试程序位于 SDK 根目录下：

development/Ingenic-Mpp/test/img2D-example

代码功能说明：

代码文件	功能
drawbox_test.c	在 Camera 数据上绘制 4 个半边框图形并通过 LCD 显示。仅 2500 支持。